

# Individual Accomplishment and Contribution

## Ong Qing Quan

Group 40 · Lumina

Teammate: Agustinus Benyamin Prasetyo

Lumina takes a batch of personal photos, groups them into events and people, and picks a best shot of each person at each event. Ben's Semester 1 measurements set out what the Semester 2 model had to change. I designed how that model works inside the application, built the backend that runs it, and tested, containerised and deployed the system.

## Model Integration

The pipeline (`lumina_pipeline.py`, `curation.py`) runs seven models on every photo: ArcFace for faces, YOLOv8 for people, OSNet for body re-identification, DINOv3 for scene, CLIP for semantics, NIMA for aesthetics and MediaPipe for blinks. I wrote the code that turns their outputs into one decision.

- **Identity.** Faces are matched one-to-one to person boxes by containment and a head-position prior, so overlapping people in a group shot stay separate. ArcFace and OSNet embeddings live in different spaces, so instead of averaging them I concatenate the two blocks scaled by  $\sqrt{\alpha}$  and  $\sqrt{1-\alpha}$ . Cosine similarity in that space is then exactly the  $\alpha$ -weighted blend of face and body similarity, and a missing modality drops out cleanly.  $\alpha$  rises with detection confidence and face size from a floor of 0.7. The floor matters: a face with no matched body gets  $\alpha = 1$ , and a low floor would shrink its similarity to the same person's face in another photo enough to split one person into two clusters. At 0.7 the worst-case shrink is about 0.84, which the clustering threshold absorbs. Faceless detections join the nearest person by body appearance, and a collection with no faces at all falls back to HDBSCAN on body embeddings.
- **Events.** The distance between two photos blends scene and time,  $0.65 \times (1 - \text{DINOv3 cosine}) + 0.35 \times \min(|\Delta t| / 6 \text{ h}, 1)$ , and falls back to scene alone when a photo has no capture time. Who appears in a photo is deliberately left out, so an identity mistake cannot redraw an event.
- **Ranking.** The seven signals are min-max normalised within each event, but each with a minimum span, so a tiny difference inside a burst of near-identical frames cannot decide the winner. A frame with eyes-open below 0.45 ranks after every open-eyes frame. Each person's best shot is re-scored on that person's own face, not the best face in the photo, and every pick carries an explanation of why it beat the runner-up.
- **Preference learning.** A "use this one instead" swap becomes one Bradley–Terry observation. The update is an SGD step with an L2 pull toward the default weights and a projection back onto the simplex, so a few noisy swaps nudge the ranker rather than overwrite it.
- **Corrections.** Renaming, merging, splitting, moving and re-pinning act on the stored result and are logged, so the log can later serve as labels.

I also built an `identity_mode` switch (fused, face only, body only) into the pipeline, which is what let the evaluation compare the three modes on the same code the app runs.

## Running the Models as a Service

The FastAPI server loads every model once at startup and runs jobs on a single worker queue, because the model objects are shared and not thread-safe. Progress streams to the browser over Server-Sent Events. Inference is batched per model (YOLO 8, DINOv3 12, CLIP and NIMA 16, OSNet 32) under `torch.inference_mode`, and the same code runs on GPU or CPU.

Per-image features are cached in SQLite under the file's SHA-256, salted with a cache version, so a repeated photo is never analysed twice and a pipeline change invalidates old entries instead of reusing them. A quick mode runs CLIP only, for users who want search without full analysis. It reads the full cache but never writes CLIP-only entries, which would hide the face features a later full run needs.

## Testing and Evaluation

I wrote the backend test suite: 207 pytest tests, all passing. They cover the fusion maths, face-to-person matching, floored normalisation, highlight selection, explanations, preference updates, corrections, the store, the pipeline's scoring path, quick mode and security, plus the API end to end against a mocked pipeline.

During development I built the first evaluation harness (`backend/eval/eval_*.py`) against the proposal targets: identity and event ARI and NMI against a labelled set, CLIP search Recall@K, cold against cached runtime, and preference learning on synthetic users with hidden weights to check that the update rule converges. The paper reports Ben's later harness rather than these scripts, but they are how the model was checked while it was being built.

## CI/CD and Deployment

GitHub Actions runs the backend unit tests and the frontend typecheck and build on every push and pull request. The CI job installs only NumPy and pytest, so it runs in seconds. The tests that need PyTorch and MediaPipe skip themselves there and run locally.

The app ships as one Railway service from a three-stage Dockerfile: Node builds the web app, a Python stage compiles InsightFace and HDBSCAN, and a slim runtime image copies in only the finished environment. FastAPI serves the app and the API from one domain. The first image was over Railway's size limit. I brought it under by installing CPU-only PyTorch (the default wheel bundles several GB of CUDA a CPU container never uses), keeping compilers out of the final image, and downloading the 2.5 GB of model weights at boot instead of baking them in. The health check allows ten minutes for that download, the service restarts on failure, and it runs exactly one replica because job progress is held in memory.

## Security

The deployed service is public. Signed-in users are identified by a verified Supabase token and anonymous users by a client id, and job endpoints check that the caller owns the job. There is a per-client rate limit, an optional shared API key, a Content Security Policy and hardened response headers. These run as pure-ASGI middleware rather than FastAPI's default middleware class, which had doubled the time to receive an upload. Expired jobs and their photos are deleted by an hourly sweep. When a user enhances a photo with an external image model, the result is checked with the same ArcFace model the pipeline uses, and a result whose faces no longer match the original is discarded.